

SISTEMA DE GESTÃO DE ESTOQUE REGIONAL

Logística • Engenharia de Algoritmos • Porto de Manaus

Equipe:

- Glleydson Meirelles
- Jones Nunes
- Gustavo Xavier

1. CONTEXTO LOGÍSTICO

O escoamento de insumos agrícolas no **Porto de Manaus Moderna** e na **Feira da Banana** apresenta gargalos logísticos tradicionais:

- **Alta Rotatividade:** Insumos como Açaí, Cupuaçu e Farinhas demandam atualização ágil de dados.
- **Risco de Ruptura:** A falta de monitoramento sobre níveis críticos inviabiliza compras preventivas.
- **Informalidade Financeira:** Dificuldade do comerciante em mapear o valor monetário imobilizado.

O Escopo do Projeto

A aplicação separa de forma estrita o **cadastro do produto** de sua **movimentação de quantidade (QTD)**, provendo consistência algorítmica e análise de negócios em tempo real.

2. ESTRUTURAS: LISTAS VS DICIONÁRIOS

≡ Listas Sequenciais

Indexadas por posições fixas de ordem linear (\$0, 1, 2...\$).

Gargalo Algorítmico: Localizar um item exige varredura elemento por elemento. Complexidade Temporal **$O(n)$** , gerando latência em inventários grandes.

🗄 Dicionários (Tabelas Hash)

Organização orientada por pares de correspondência direta **Chave: Valor**.

Acesso Instantâneo: A recuperação do registro ocorre via acoplamento direto de chave única. Complexidade Temporal **$O(1)$** , performando em tempo constante.

3. ARQUITETURA DO BANCO DE DADOS

```
# Estrutura de Dicionários Aninhados
estoque = {
    1: {"nome": "Açaí", "quantidade": 20, "preco": 15.00},
    2: {"nome": "Cupuaçu", "quantidade": 10, "preco": 12.50},
    3: {"nome": "Farinha", "quantidade": 30, "preco": 8.00},
    4: {"nome": "Castanha", "quantidade": 5, "preco": 25.00}
}

# Ponteiro incremental automático
proximo_codigo = 5
```

Mapeamento Baseado em Chaves Primárias

- A chave do dicionário externo atua como uma **Chave Primária Numérica (ID)**, isolando a identidade lógica do produto.
- Cada **ID** armazena internamente as propriedades de tipo primitivo (`nome`, `quantidade` e `preco`).
- A variável `proximo_codigo` gerencia de forma global os incrementos e novas entradas sem colisões.

4. FUNDAMENTAÇÃO TEÓRICA: AS OPERAÇÕES CRUD

O padrão **CRUD** governa as quatro operações fundamentais de persistência e ciclo de vida de dados estáveis:

Create

Alocação inicial e injeção do novo registro de produto.

Read

Varreduras gerais e consultas filtradas do inventário.

Update

Alteração cirúrgica in-place (Incremento de fardos).

Delete

Expurgo ou desalocação física do registro na memória.

5. CRUD: CADASTRO DE PRODUTO (CREATE)

```
def cadastrar_novo_produto(estoque_local):  
    global proximo_codigo  
    nome = input("Nome: ").strip().title()  
  
    for dados in estoque_local.values():  
        if dados["nome"] == nome:  
            print("⚠ Já cadastrado!"); return  
  
    try:  
        preco_in = input("Preço: ").strip()  
        preco = float(preco_in.replace(",", "."))  
    except ValueError:  
        print("❌ Inválido!"); return  
  
    estoque_local[proximo_codigo] = {  
        "nome": nome,  
        "quantidade": 0
```

Sanitização e Validação Algorítmica

- A chave do dicionário externo atua como uma **Chave Primária Numérica (ID)**, isolando a identidade lógica do produto.
- Cada **ID** armazena internamente as propriedades de tipo primitivo (`nome`, `quantidade` e `preco`).
- A variável `proximo_codigo` gerencia de forma global os incrementos e novas entradas sem colisões.

6. CRUD: BUSCA HÍBRIDA E CONSULTAS (READ)

```
def buscar_produto(estoque_local):  
    opcao = input("1-Código ou 2-Nome: ").strip()  
    if opcao == "1":  
        cod = int(input("Código: "))  
        if cod in estoque_local:  
            print(estoque_local[cod])  
    elif opcao == "2":  
        nome = input("Nome: ").strip().title()  
        for cod, p in estoque_local.items():  
            if p["nome"] == nome:  
                print(p); break
```

Estratégia de Busca Multinível

- **Mapeamento Indexado:** A consulta direta por ID numérico responde na velocidade máxima do dicionário.
- **Varredura Linear:** Busca nominal estruturada usando `.items()`.
- O laço nominal é otimizado com a instrução `break` para encerrar a iteração imediatamente ao localizar a meta.

7. CRUD: MOVIMENTAÇÃO DE QUANTIDADE (UPDATE)

Mutação In-Place por Chave Primária

- A busca pelo produto é direta a partir do ID numérico informado, operando na máxima velocidade da tabela hash $O(1)$.
- Valida a existência do ID na coleção raiz usando o operador de associação in.
- **Operador de Acumulação:** Aplica o modificador composto += para somar a nova carga sem apagar o histórico físico anterior do item.

```
def adicionar_estoque_por_codigo(estoque_local):  
    codigo = int(input("Digite o CÓDIGO: "))  
  
    if codigo in estoque_local:  
        produto = estoque_local[codigo]  
        qtd = int(input("Quantidade de entrada: "))  
        if qtd < 0: return  
  
        # Modificação in-place via acumulador  
        produto["quantidade"] += qtd
```


8. CRUD: EXCLUSÃO DE REGISTROS (DELETE)

Desalocação Atômica de Memória RAM

- Utiliza o método nativo `.pop(código)` das tabelas mapeadas em Python.
- Remove fisicamente a chave primária informada e todos os seus dicionários internos de dados.
- Garante que qualquer cálculo financeiro ou relatório logístico emitido em seguida ignore por completo o item expurgado.

```
def remover_produto(estoque_local):  
    codigo = int(input("Código para remover: "))  
  
    if codigo in estoque_local:  
        # Expurgo físico imediato da memória  
        removido = estoque_local.pop(codigo)  
        print(f"✅ {removido['nome']} removido.")
```

9. LÓGICA CONDICIONAL: LIMIAR DE ESTOQUE BAIXO

Filtros por Restrição Linear

- A emissão do relatório logístico utiliza desvios condicionais baseados em operadores lógicos relacionais ($\leq$).
- O parâmetro limiar é fixado em **10 unidades** padrão, mantendo-se parametrizável.
- A flag booleana de controle desvia o fluxo para exibir mensagens de conformidade caso nenhum item viole o limiar.

```
encontrou = False
for codigo, dados in estoque_local.items():
    # Teste de desigualdade relacional
    if dados["quantidade"] <= limiar:
        print(f"• {dados['nome']}: {dados['quantidade']}")
        encontrou = True
```

10. LAÇOS DE REPETIÇÃO E ACUMULADORES FINANCEIROS

Consolidação de Patrimônio em Estoque

O cálculo matemático automático do capital alocado baseia-se no conceito algorítmico de redução vetorial por acumulador:

- **Expressão Geradora Compacta:** O laço percorre os subconjuntos de valores de forma implícita e veloz através de uma generator expression.
- A função embarcada `sum()` centraliza a responsabilidade do acumulador numérico, computando em tempo real a soma algébrica:
$$\sum_{i=1}^n (\text{Quantidade}_i \times \text{Preço}_i)$$

```
def calcular_valor_total(estoque_local):  
    # Redução sum com laço generator implícito  
    total = sum(  
        d["quantidade"] * d["preco"]  
        for d in estoque_local.values()  
    )  
    print(f"💰 Total Patrimonial: R$ {total:.2f}")
```

11. INTERFACE DE USUÁRIO E MENU EM LOOP WHILE

```
while True:
    exibir_menu_cli()
    opcao = input("Escolha: ").strip()

    if opcao == "1": cadastrar_novo_produto(estoque)
    elif opcao == "0": break
```

Persistência Lógica e Máquina de Estados

- A instrução de loop infinito **while True** mantém a sessão de balcão ativa por tempo indeterminado, preservando o programa em memória.
- Direciona tarefas via despacho centralizado guiado por blocos encadeados **if-elif-else**.
- A cláusula **break** interrompe a persistência e encerra o processo de forma limpa.

12. INTELIGÊNCIA ARTIFICIAL: SISTEMA ESPECIALISTA

Tomada de Decisão Baseada em Regras

O módulo analítico de sugestões de compra (Opção 8) adota as diretrizes clássicas da **IA Simbólica**:

- Estrutura uma árvore de decisão determinística alimentada por condicionais lineares encadeados.
- **Métrica Cruzada:** Cruza riscos físicos de gôndola (estoque crítico ≤ 5) com indicadores de capital retido ($\leq R\$ 500,00$), blindando a liquidez do negócio.

```
if dados["quantidade"] <= 5:  
    print("❌ COMPRAR IMEDIATAMENTE (Crítico)")  
elif valor_investido > 500:  
    print("💰 CAPITAL RETIDO (Risco de liquidez)")  
else:  
    print("✅ ESTOQUE ADEQUADO")
```

13. GOVERNANÇA DE CÓDIGO NO GITHUB



``estoque_regional.py``

Script estruturado de forma desacoplada em blocos limpos, com nomenclaturas inteligíveis.



``README.md``

Documentação de alto nível detalhando a arquitetura $O(1)$, equações lógicas e manuais de execução.



``requirements.txt``

Garantia de conformidade ambiental com foco estratégico no uso performático da Biblioteca Padrão.

14. CONCLUSÕES FINAIS DO PROJETO

Resultados Logísticos

O sistema mitiga falhas de preenchimento e mapeia instantaneamente o estoque em nível crítico de desabastecimento

Proteção De Liquidez

A árvore de decisão da IA impede o acúmulo desnecessário de capital estagnado, balanceando o fluxo de caixa do pequeno feirante.

Rigor Acadêmico

O projeto valida a aplicação prática dos conceitos fundamentais de desenvolvimento (Operações CRUD, tabelas hash e lógica simbólica) aplicados em um arranjo produtivo regional.

MUITO OBRIGADO!